

HelixAgent

HelixAgent

Tek bir model seçmeyin — onların tartışmasını sağlayın ve üzerinde uzlaştıkları yanıtı sunun.

Özet

HelixAgent, Go tabanlı, AI destekli bir topluluk (ensemble) LLM hizmetidir. Birden fazla dil modelinden gelen yanıtları — çok turlu AI tartışma sistemi ve dinamik doğrulama tabanlı sağlayıcı seçimi dahil — akıllıca birleştirerek en doğru ve güvenilir çıktıyı üretir.

Kısa Açıklama

HelixAgent, Go tabanlı bir topluluk LLM hizmetidir ve birçok sağlayıcıyı tek bir doğru yanıtta birleştirir. Çok turlu AI tartışmaları yürütür, sağlayıcıları LLMsVerifier aracılığıyla dinamik olarak puanlar, güven ağırlıklı yönlendirme stratejileri kullanır ve üretim özellikleri sunar: önbellekleme, izleme, güvenlik korumaları ve OpenAI tarzı API'ler.

Uzun Açıklama

HelixAgent, üretime hazır, AI destekli bir topluluk LLM hizmetidir (MIT lisanslı). Tek bir modelin yanıtını kesin bir hüküm olarak değil, bir hipotez olarak ele alır. Sonucu tek bir sağlayıcıya bağlamak yerine — ki bu sağlayıcı yanlış, önyargılı veya geçici olarak erişilemez olabilir — birden fazla dil modelinden gelen yanıtları birleştirerek en doğru ve güvenilir çıktıya ulaşır. Soru yeterince zorlayıcı olduğunda ise modelleri yapılandırılmış, çok turlu bir tartışma sürecinden geçirir. Kapsamı geniştir: README dosyasında `internal/llm/providers/` altında birçok LLM sağlayıcısı belgelenmiştir; bunlar arasında Claude, DeepSeek, Gemini, Mistral, Qwen ve xAI/Grok yer alır.

Önemli olan, sağlayıcı seçiminin statik bir tercih listesi olmamasıdır — bu seçim gerçek zamanlı olarak kazanılır. Entegre LLMsVerifier’den gelen canlı doğrulama puanları, yönlendirmeyi ve en iyi performans gösteren sağlayıcıya sorunsuz geçişi sağlar; bir sağlayıcıda performans düşüşü olduğunda ise kategorilendirilmiş hata raporlaması devreye girer. AI Tartışma Orkestratörü, anlaşmazlıkları sinyale dönüştürür: Birden fazla topoloji (ağ, yıldız, zincir) ve disiplinli bir aşama protokolü destekler — Öneri → Eleştiri → Değerlendirme → Sentez — ve sistem, modeller arasındaki uzlaşmayı zamanla iyileştiren çapraz tartışma öğrenimiyle donatılmıştır. Yönlendirme stratejileri, güven ağırlıklı seçim, çoğunluk oyu uzlaşısı ve anlamsal niyet tespitini kapsar; tüm bunlar gerçek zamanlı akışlı yanıtlarla desteklenir, böylece yanıtlar tüm topluluğun karar vermesini beklemek yerine jeton jeton gelir.

Hizmet, yalnızca demo amaçlı değil, üretim ortamında hayatta kalacak şekilde tasarlanmıştır: PostgreSQL ve Redis yüksek erişilebilirlikli bir veri katmanı oluştururken, Prometheus/Grafana/OpenTelemetry metrikler, kontrol panelleri ve izleme sağlar. JWT kimlik doğrulama, hız sınırlama, koruma mekanizmaları ve PII tespiti, topluluğu gerçek bir dağıtımın gerektirdiği kontrollerle sarar. Yaklaşık yirmi ayrıştırılmış modülden (EventBus, Observability, Auth, Storage, VectorDB, Embeddings, RAG, Memory, MCP ve diğerleri) oluşur; her biri bağımsız bir işlevi yerine getirir. Ayrıca, LLM optimizasyon çerçevesi (anlamsal önbellekleme, yapılandırılmış çıktı, geliştirilmiş akış) ve SGLang, LlamaIndex, LangChain, Guidance ve LMQL için entegrasyonlar sunar. Tamamlama ve topluluk uç noktaları OpenAI uyumlu olduğundan, mevcut bir istemci HelixAgent’a yönlendirilerek topluluk tabanlı akıl yürütme elde edilebilir — yeniden yazım gerektirmeden.

İçerik

Neden inşa ettik

Tek bir LLM hatalı, önyargılı veya erişilemez olabilir. HelixAgent, uygulamaların aynı anda birçok modelden yararlanabilmesi, yanıtlarını ölçülen güvenilirliklerine göre tartabilmesi ve sorunsuz bir şekilde yedeklenebilmesi için geliştirildi — tek bir sağlayıcıya bağımlı kırılgan bir yapıyı, kendi kendini puanlayan dayanıklı bir topluluğa dönüştürmek için.

Neden oyunun kurallarını değiştiriyor

Çoklu model uzlaşısını işlevselleştiriyor — “birkaç modele sorup uzlaştır” yaklaşımını geçici betiklerden çıkarıp üretim hizmetine taşıyor. Tek bir sağlayıcıya sabit kodlanıp umut etmek yerine,

ekipler canlı doğrulama puanlarıyla yönlendirilen yönlendirme, tek seferlik yanıtın yeterli olmadığı sorular için yapılandırılmış bir tartışma protokolü ve üretim düzeyinde dayanıklılık (yüksek erişilebilirlikli veri katmanı, tam gözlemlenebilirlik ve koruma mekanizmaları) elde ediyor; tüm bunlar OpenAI uyumlu bir API arkasında sunuluyor. Asıl kazanç, kesintisiz benimseme: Tek bir kırılğan sağlayıcı bağımlılığı, dayanıklı ve kendi kendini puanlayan bir topluluğa dönüşüyor; mevcut istemciler kodu değiştirmek yerine sadece bir uç noktayı güncelleyerek geçiş yapabiliyor.

Yenilikçi olan ne

- Model anlaşmazlığını bir kaynak olarak gören yapılandırılmış çok turlu AI tartışması: Seçilebilir ağ/yıldız/zincir topolojileri, disiplinli bir Öneri→Eleştiri→İnceleme→Sentez protokolü ve zaman içinde biriken tartışmalar arası öğrenme.
- Statik bir tercih listesinden ziyade canlı LLMsVerifier puanlarından elde edilen dinamik sağlayıcı seçimi — topluluk, şu anda gerçekten performans gösteren modele yönlendirme yapıyor ve biri düşüşe geçtiğinde sorunsuz bir şekilde yedekleniyor.
- Kendi başına ayakta durabilen, isteğe bağlı olarak dış optimizasyon araçları (SGLang, LlamaIndex, LangChain, Guidance, LMQL) eklenebilen yerel bir Go LLM optimizasyon çerçevesi (anlamsal önbellek, yapılandırılmış çıktı, geliştirilmiş akış).
- Yaklaşık yirmi ayrıştırılmış modülden oluşan modüler bir mimari; bu sayede endişeler ayrıştırılabilir ve dağıtık bellek, bilgi grafiği akışı gibi Büyük Veri özelliklerine kapı aralanıyor.

En büyük teknik zorluklar ve çözümlerimiz

- **Birçok eşit olmayan sağlayıcı arasından seçim yapmak.** Sağlayıcılar kalite açısından farklılık gösterir ve zaman içinde değişir; bu nedenle sabit bir sıralama yarından itibaren geçersiz olur. Çözümümüz, seçimi sürekli ölçülebilir hale getirmek oldu: LLMsVerifier puanları, güven ağırlıklı ve çoğunluk oylamalı yönlendirmeyi besliyor; böylece performansı düşen bir sağlayıcıya güvenmek yerine etrafından dolaşılıyor.
- **Gerçekten zor sorulara güvenilir yanıt almak.** Tek bir model, tek seferde sorulduğunda kendi hatasını yakalayacak bir mekanizmaya sahip değil. Tartışma Orkestratörü bunu sağlıyor — çoklu topoloji, aşamalı tartışma (Öneri → Eleştiri → İnceleme → Sentez) modellerin birbirini sorgulayıp geliştirmelerini zorunlu kılıyor, nihai yanıt sentezlenmeden önce.

- **Bir topluluğu sadece bir not defterinde değil, üretim ortamında çalıştırmak.** Birçok sağlayıcıya dağıtım, arıza yüzeyini katlıyor. Bunu PostgreSQL+Redis yüksek erişilebilirlikli veri katmanını, sağlayıcı veya yönlendirme hatalarında devreye giren Prometheus/Grafana/OpenTelemetry gözlemlenebilirliği ve JWT kimlik doğrulama, hız sınırlaması, koruma motoru ile PII tespiti içeren bir güvenlik çevresiyle kontrol altına aldık.

İçerik

Teknoloji Yığını

- **Go** — Tek bir isteği eşzamanlı olarak birçok sağlayıcıya dağıtmak tam da gorutin'lerin yaptığı iş olduğu için seçildi; tek bir ikili dosya halinde dağıtım, ~20 modüllük servisin gönderimini basit tutuyor. Tüm servisin ve her bir iç modülün temelini oluşturuyor.
- **Gin (Web API)** — Hızlı ve düşük ek yük sunan bir HTTP arayüzü için seçildi; mevcut istemcilerin topluluğu değiştirmeden benimseyebilmesini sağlayan, OpenAI uyumlu / v1 tamamlama, sohbet, akış ve topluluk uç noktalarını sunuyor.
- **PostgreSQL** — Oturumlar, analizler ve tartışma kayıtları için kalıcı depolama olarak seçildi; böylece fikir birliği kararları ve tartışma geçmişi denetlenebilir hale geliyor. Yüksek erişilebilirlik veri katmanının temelini oluşturuyor.
- **Redis** — Düşük gecikmeli önbellekleme ve görev sıralama için seçildi; hem yanıt önbelleklemeyi hem de tekrarlanan veya neredeyse aynı olan istemlerin gereksiz çıkarım işlemlerini atlamasını sağlayan anlamsal önbellek katmanını güçlendiriyor.
- **LLMsVerifier (entegre)** — Sağlayıcı güvenilirliğini varsayım olmaktan çıkarıp ölçülebilir bir niceliğe dönüştürmek için seçildi; puanları, yönlendirme için sağlayıcıları sıralıyor ve bir sağlayıcının performansı düştüğünde yedeklemeyi tetikliyor.
- **Prometheus + Grafana + OpenTelemetry** — Birçok sağlayıcıyı kapsayan bir topluluğun gözlemlenebilir kalması için seçildi; helixagent_* metriklerini, kontrol panellerini ve isteğin tüm dağıtım sürecindeki uçtan uca izlemeyi sunuyorlar.
- **Model Context Protocol (MCP) adaptörleri** — Açık bir protokol aracılığıyla genişletilebilirlik için seçildi; README dosyasında, harici araçlar ve bağlamlarla bağlantı kurmak için birçok MCP adaptörü listeleniyor.
- **Neo4j / ClickHouse / Kafka (Büyük Veri)** — Tek bir düğümün ötesine geçmek için seçildi: Neo4j ve ClickHouse, dağıtık bellek ve bilgi grafiği özelliklerini desteklerken, Kafka bu grafiği ve olay verilerini ölçekli olarak aktarıyor.
- **Optimizasyon entegrasyonları (SGLang, LlamaIndex, LangChain, Guidance, LMQL)** — Ön ek önbellekleme, bilgi getirme, görev ayrıştırma ve kısıtlı üretim gibi isteğe bağlı

hizmetler olarak eklenebilmeleri için seçildi; böylece daha ağır optimizasyonlar mevcut olurken zorunlu olmuyor.

Durum ve Dürüstlük Notları

- **Durum: beta.** Servis, üretim için hazır olarak tanımlanıyor, ancak README dosyasındaki performans ve kapsam rakamları (ör. “1000+ istek/saniye”, “<500ms önbellekli”, sağlayıcı ve doğrulama betiği sayıları) proje tarafından kendi beyanı olup bağımsız olarak doğrulanmamıştır ve burada kasıtlı olarak nitel ifadelerle aktarılmıştır.
- README dosyası içindeki sağlayıcı sayıları da değişkenlik gösteriyor; sayfa, “çok sayıda sağlayıcı” şeklinde nitel bir çerçeve kullanıyor.

Öncelikli katman: Helix-birincil.